



Almacenamiento de datos

Preferencias Compartidas

Preferencias compartidas, o shared preferences, es la forma que tiene Android para almacenar y recuperar pequeñas cantidades de datos primitivos como pares de clave/valor de un archivo en el almacenamiento del dispositivo como String, int, float, boolean que componen sus preferencias en un archivo XML dentro de la aplicación en el almacenamiento del dispositivo. **Las Preferencias Compartidas pueden considerarse como un diccionario o un par clave/valor.**

Las Preferencias compartidas son adecuadas para diferentes situaciones. Por ejemplo, cuando la configuración del usuario necesita ser guardada o almacenar datos que se pueden utilizar en diferentes actividades dentro de la aplicación. Para que los datos se guarden persistentemente, se prefiere guardarlo en el método onPause(), que podría ser restaurado en la actividad onResume ().

Los datos almacenados mediante preferencias compartidas se mantienen privados en el ámbito de aplicación.

Una aplicación Android puede gestionar varias colecciones de preferencias, que se diferenciarán mediante un identificador único. Para obtener una referencia a una colección determinada utilizaremos el método getSharedPreferences() al que pasaremos el identificador de la colección y un modo de acceso.

~~El modo de acceso indicará qué aplicaciones tendrán acceso a la colección de preferencias y qué operaciones tendrán permitido realizar sobre ellas. Así, tendremos tres posibilidades principales:~~

- ~~• MODE_PRIVATE. Sólo nuestra aplicación tiene acceso a estas preferencias.~~
- ~~• MODE_WORLD_READABLE. Todas las aplicaciones pueden leer estas preferencias, pero sólo la nuestra puede modificarlas.~~
- ~~MODE_WORLD_WRITEABLE. Todas las aplicaciones pueden leer y modificar estas preferencias.~~

Anteriormente, Android soportaba varios modos (como MODE_WORLD_READABLE o MODE_WORLD_WRITEABLE), que permitían el acceso de otras aplicaciones, pero estos modos han sido obsoletos por motivos de seguridad. Actualmente, solo está disponible



`MODE_PRIVATE`, que hace que las preferencias sean accesibles únicamente por la aplicación que las creó.

Para acceder a una colección específica, se utiliza el método `getSharedPreferences()`, en el que se debe pasar: Identificador de la colección y el Modo de acceso

```
SharedPreferences prefs =  
    getSharedPreferences("Preferencias",Context.MODE_PRIVATE);
```

```
val prefs = getSharedPreferences("Preferencias", MODE_PRIVATE)
```

Lectura: Para recuperar un valor, se utiliza un método de tipo `get` (como `getString`, `getInt`, `getBoolean`, etc.) pasando la clave y un valor por defecto en caso de que no se encuentre la clave.

```
String nombre = prefs.getString("nombre", "Daniel");
```

```
val nombre = prefs.getString("nombre", "Daniel")
```

Escritura: Para actualizar o insertar nuevas preferencias el proceso será igual de sencillo, con la única diferencia de que la actualización o inserción no la haremos directamente sobre el objeto `SharedPreferences`, sino sobre su objeto de edición `SharedPreferences.Editor`, donde se aplican los cambios con `apply()` o `commit()`.

```
SharedPreferences.Editor editor = prefs.edit();  
editor.putString("nombre", "Manuel");  
editor.commit();
```

```
val editor = prefs.edit()  
editor.putString("nombre", "Manuel")  
editor.apply()
```

```
prefs.edit {  
    putString("nombre", "Manuel")  
}
```

¿Pero donde se almacenan estas preferencias compartidas?

`/data/data/paquete.java/shared_prefs/nombre_coleccion.xml`

Ej: `/data/data/es.damiguet.dam2.datos/shared_prefs/nombre_coleccion.xml`

no es recomendable para almacenar información sensible



```
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/main"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textview"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="30dp"
        android:text="Shared Preferences Perfil"
        android:textSize="24sp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <EditText
        android:id="@+id/editTnombre"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="50dp"
        android:hint="Introduce tu nombre"
        android:padding="10dp"
        app:layout_constraintTop_toBottomOf="@id/textview" />

    <EditText
        android:id="@+id/editTemail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@+id/editTnombre"
        android:layout_marginTop="20dp"
        android:hint="Introduce tu correo"
        android:inputType="textEmailAddress"
        android:padding="10dp"
        app:layout_constraintTop_toBottomOf="@+id/editTnombre"
        tools:layout_editor_absoluteX="32dp" />
```



```
</androidx.constraintlayout.widget.ConstraintLayout>
```

```
override fun onResume() {  
    super.onResume()  
    val shPref = getSharedPreferences("Preferencias", MODE_PRIVATE)  
    val spNombre = shPref.getString("nombre", "")  
}  
  
override fun onPause() {  
    super.onPause()  
    val shPref = getSharedPreferences("Preferencias", MODE_PRIVATE)  
    shPref.edit {  
        putString("nombre", nombre.text.toString())  
    }  
}
```

Jetpack DataStore

Reemplaza SharedPreferences con API basada en Kotlin Coroutines y Flow.

Jetpack DataStore es una solución de almacenamiento de datos que te permite almacenar pares clave-valor o bien objetos escritos con búferes de protocolo. Datastore usa corrutinas y Flow de Kotlin para almacenar datos de manera asíncrona, coherente y transaccional.

Si usas SharedPreferences para almacenar datos, considera migrar a DataStore.

Agregar la dependencia:

```
implementation("androidx.datastore:datastore-preferences:1.2.0")
```

```
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.10.0")
```



DataStore usa MVVM con Flow

Crea un archivo llamado SharedPreferencesViewModel

Se recomienda usar by preferencesDataStore (es un singleton automático) :

```
import android.content.Context
import androidx.datastore.preferences.preferencesDataStore

val Context.dataStore by preferencesDataStore(name = "settings")
```

"settings" es el nombre del archivo, similar a SharedPreferences.

Genera un archivo DataStore llamado "settings"

Asegura que solo exista una instancia, sin riesgo de duplicados

Haz que la nueva clase implemente ViewModel()

```
class SharedPreferencesViewModel: ViewModel() {
```

inicializa todas las keys que necesites

```
companion object {
    val NOMBRE_KEY = stringPreferencesKey("nombre")
    val EDAD_KEY = intPreferencesKey("edad")
}
```

o utiliza constantes

```
val HORA_KEY = stringPreferencesKey("hora")
```

Crea tantos getter/setter como necesites.

Ejemplo getter

```
fun getNombre(context: Context): Flow<String> {

    return context.dataStore.data.map { preferences ->
        preferences[NOMBRE_KEY] ?: "Sin nombre"
    }
}
```

Ejemplo Setter

```
fun setNombre(context: Context, nombre: String) {
    viewModelScope.launch {
```



```
context.dataStore.edit { preferences ->
    preferences[NOMBRE_KEY] = nombre
}
}
```

*edit es suspend function, se debe llamar desde una coroutine o viewModelScope.launch.

Si nos movemos a nuestra actividad ya podremos utilizar nuestro ViewModel

Instanciamos nuestro ViewModel

```
val viewModelSP =
    ViewModelProvider(this) [SharedPreferencesViewModel::class.java]
```

Getter

```
lifecycleScope.launch {
    viewModelSP.getNombre(this@MainActivity).collect { name ->
        nombre.setText(name)
    }
}
```

Setter

```
viewModelSP.setNombre(this@MainActivity, "Daniel")
```